

Clase Práctica : Capa de aplicación

Tomás F. Melli

November 2025

Índice

1	Introducción	2
1.1	Protocolos de la Capa de Aplicación	2
1.2	Aplicaciones y Tipo de Servicio de las Capas Inferiores	2
1.3	Arquitecturas de Comunicación	2
1.4	Aplicación versus Protocolo	2
2	HTTP (Hypertext Transfer Protocol)	3
2.1	Peticiones HTTP	3
2.1.1	Estructura	3
2.1.2	Ejemplo	4
2.2	Respuestas HTTP	4
2.2.1	Estructura	4
2.2.2	Ejemplo	5
2.3	Encabezados HTTP Esenciales	5
3	Ejercicio 1	5
4	Versiones del Protocolo HTTP	6
4.1	HTTP/1.0 — RFC 1945	6
4.1.1	Características principales	6
4.2	HTTP/1.1 — RFC 2616	6
4.2.1	Principales mejoras	7
4.2.2	Problemas de HTTP/1.1	7
4.3	HTTP Persistente y Pipelining	7
4.4	Etiquetas HTML Relevantes	8
4.5	Técnicas de Cacheo en HTTP	8
5	Ejercicio 2	9
6	Correo Electrónico (SMTP, POP3, IMAP, Webmail, MIME)	10
6.1	Protocolos de correo	11
6.2	Ejemplo de flujo SMTP	11
6.3	Formato del mensaje	11
6.4	Extensiones MIME (Multipurpose Internet Mail Extensions)	11
6.5	Ejemplo de extensiones al formato de mensaje	12
6.6	POP3 e IMAP — Recepción de mensajes	12
6.6.1	POP3 — Post Office Protocol v3	12
6.6.2	IMAP — Internet Message Access Protocol	12
6.6.3	Webmail	12
7	Ejercicio 3	13

1 Introducción

En la capa de aplicación, las aplicaciones de red hacen uso de protocolos ya establecidos para la comunicación entre procesos distribuidos. Estos protocolos definen **reglas precisas de intercambio de información** que permiten la interoperabilidad entre distintos sistemas y plataformas. Cada protocolo especifica la **sintaxis** (formato) y la **semántica** (significado) de los mensajes transmitidos.

1.1 Protocolos de la Capa de Aplicación

Los protocolos de la capa de aplicación son aquellos diseñados para servicios específicos, tales como:

- **HTTP (Hypertext Transfer Protocol)** para la transferencia de documentos web.
- **SMTP, POP3, IMAP** para servicios de correo electrónico.
- **FTP** para transferencia de archivos.
- **DNS** para resolución de nombres de dominio.

Estos protocolos se apoyan en los servicios que brindan las capas inferiores, como **TCP** o **UDP**, para garantizar la entrega de los datos según el tipo de servicio requerido.

1.2 Aplicaciones y Tipo de Servicio de las Capas Inferiores

Las **aplicaciones son procesos software** que se comunican a través de la red mediante protocolos de aplicación. La elección del tipo de servicio de las capas inferiores (orientado a conexión con TCP o no orientado a conexión con UDP) depende de las necesidades de la aplicación:

- Aplicaciones que requieren fiabilidad y orden (como HTTP o correo electrónico) utilizan **TCP**.
- Aplicaciones que priorizan la velocidad o toleran pérdida de datos (como DNS o streaming) utilizan **UDP**.

1.3 Arquitecturas de Comunicación

Existen varios paradigmas de comunicación en la capa de aplicación:

Paradigma Cliente-Servidor

Un proceso cliente solicita servicios y un proceso servidor los provee. Ejemplo: un navegador web (cliente) realiza peticiones HTTP a un servidor web.

Paradigma P2P (Peer-to-Peer)

Los nodos actúan simultáneamente como clientes y servidores, compartiendo recursos entre iguales. Ejemplo: redes de intercambio de archivos como BitTorrent.

Arquitecturas Híbridas

Combinan características de cliente-servidor y P2P, como ocurre en algunas aplicaciones de mensajería instantánea o plataformas de distribución de contenido.

RPC (Remote Procedure Call)

Permite que un proceso invoque funciones o procedimientos en otro sistema remoto como si fueran locales, ocultando los detalles de la comunicación subyacente.

1.4 Aplicación versus Protocolo

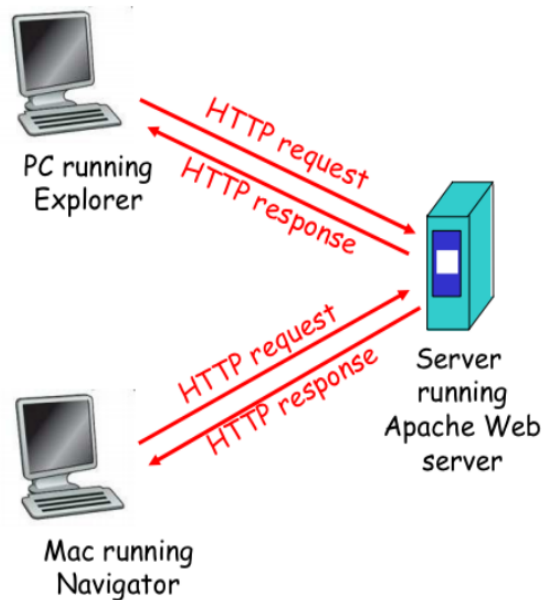
Es importante distinguir entre una **aplicación** y un **protocolo**:

- El **protocolo** define el conjunto de reglas y formatos de los mensajes intercambiados (por ejemplo, HTTP).
- La **aplicación** es el software que implementa dicho protocolo y lo utiliza para ofrecer un servicio al usuario (por ejemplo, Firefox o Chrome).

Por lo tanto, **HTTP** $\neq$ **Firefox**, ya que el primero es un protocolo y el segundo una aplicación que lo emplea.

2 HTTP (Hypertext Transfer Protocol)

HTTP es el protocolo principal de la **World Wide Web**, y se utiliza para la transferencia de recursos entre navegadores y servidores web.



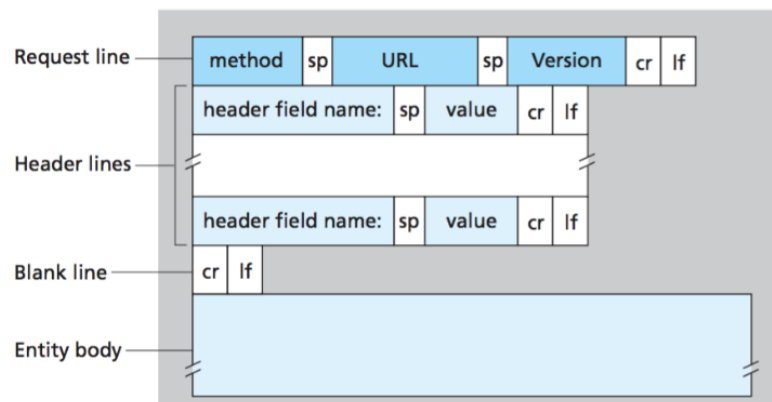
- Opera sobre **TCP**, normalmente en el **puerto 80** (o 443 en su versión segura HTTPS).
- La comunicación se realiza mediante el intercambio de **mensajes HTTP**, que incluyen un **HEADER** (encabezado con metadatos) y un **BODY** (contenido del recurso).
- Cada recurso en la web se identifica mediante una **URL (Uniform Resource Locator)**.
- HTTP es un protocolo **sin estado (stateless)**: el servidor no mantiene información sobre las conexiones previas.

2.1 Peticiones HTTP

Las peticiones HTTP son enviadas por el cliente y definen la operación que se desea realizar sobre un recurso. Los principales métodos son:

Método	Descripción
GET	Solicita un recurso o información del servidor.
HEAD	Solicita solo los encabezados de una respuesta, sin el cuerpo.
POST	Envía datos al servidor, típicamente para crear un nuevo recurso.
PUT	Envía datos para actualizar o reemplazar un recurso existente.
DELETE	Solicita la eliminación de un recurso.

2.1.1 Estructura



2.1.2 Ejemplo

Línea de requerimiento
(comandos GET, POST,
HEAD)

Líneas de encabezado

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: es
```

Carriage return,
line feed
Indica fin de mensaje

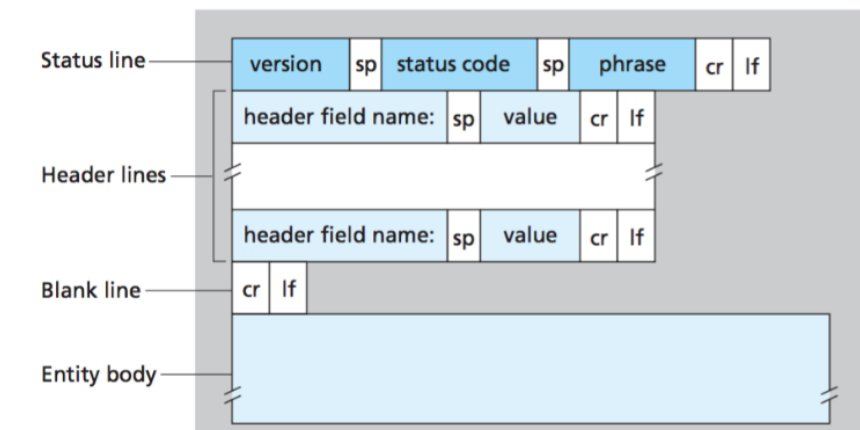
(carriage return, line feed extra)

2.2 Respuestas HTTP

Las respuestas son enviadas por el servidor e incluyen un **código de estado** que indica el resultado de la operación:

Código	Categoría	Descripción
1xx	Informativo	La solicitud fue recibida y está siendo procesada.
2xx	Exitoso	La solicitud fue procesada correctamente.
3xx	Redirección	Se requiere acción adicional para completar la solicitud.
4xx	Error del Cliente	La solicitud contiene errores (p. ej., 404 Not Found).
5xx	Error del Servidor	El servidor falló al procesar una solicitud válida.

2.2.1 Estructura



2.2.2 Ejemplo

Línea de estatus
(código de estatus
del protocolo
Frase de estatus) → HTTP/1.1 200 OK

Líneas de
encabezado → Connection close
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998
Content-Length: 6821
Content-Type: text/html

data, e.g.,
archivo
HTML solicitado → data data data data data ...

2.3 Encabezados HTTP Esenciales

Encabezado	Tipo	Descripción
Host	Petición	Nombre de dominio del servidor solicitado.
Content-Type	Ambos	Tipo de medio (ej. HTML, JSON, imagen) del contenido.
Content-Length	Ambos	Tamaño del cuerpo del mensaje en bytes.
Cache-Control	Ambos	Directivas para el control del almacenamiento en caché.
If-Modified-Since	Petición	Solicita el recurso solo si fue modificado después de cierta fecha.
Authorization	Petición	Contiene credenciales de autenticación del cliente.
Cookie	Petición	Envía cookies almacenadas por el cliente al servidor.
User-Agent	Petición	Identifica el software del cliente (navegador o aplicación).

3 Ejercicio 1

El protocolo HTTP permite hacer distintos tipos de pedidos para recursos de un determinado dominio. Escriba los Requests HTTP 1.1 que permitan obtener los siguientes pedidos al sitio web del departamento de computación:

El recurso /

Para obtener el recurso / :

```
1 GET / HTTP/1.1
2 Host: www.dc.uba.ar
```

Encabezado del recurso /tdc

```
1 HEAD /tdc HTTP/1.1
2 Host: www.dc.uba.ar
```

El recurso /logo.jpg si no fue modificado desde una determinada fecha

```
1 GET /logo.jpg HTTP/1.1
2 Host: www.dc.uba.ar
3 If-Modified-Since
```

Crear un nuevo post en el foro del departamento

```
1 POST /foro HTTP/1.1
2 Host: www.dc.uba.ar
3 Content-Type: application/json
4 Content-Length: 70
5 <datos>
```

4 Versiones del Protocolo HTTP

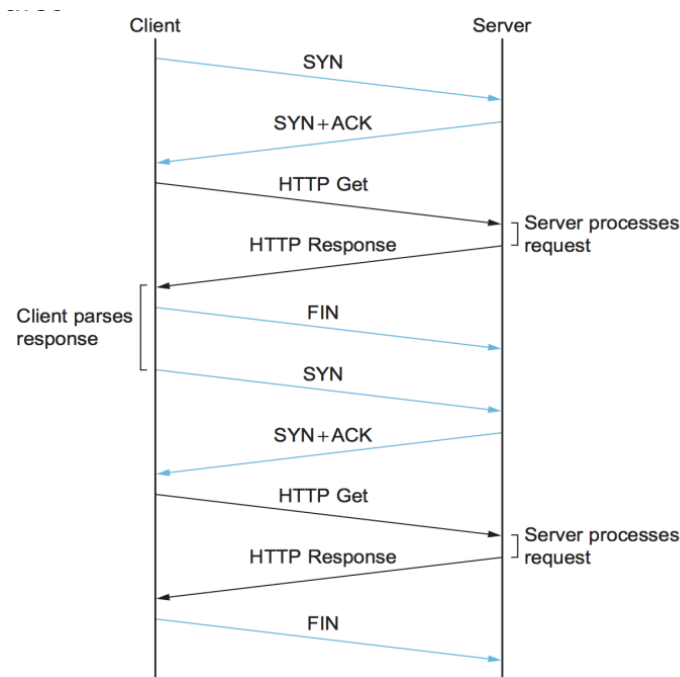
4.1 HTTP/1.0 — RFC 1945

La versión **HTTP/1.0** fue especificada en el año 1996 en el [RFC 1945](#). Representa la primera versión estandarizada del protocolo HTTP, formalizando las prácticas utilizadas por los navegadores y servidores de la época.

4.1.1 Características principales

- Utiliza el puerto 80 por defecto sobre TCP.
- Cada solicitud abre una **nueva conexión TCP**, lo que implica un alto costo de establecimiento y cierre por cada recurso solicitado.
- Define los métodos básicos: **GET**, **HEAD** y **POST**.
- Introduce los códigos de estado (1xx–5xx) y los encabezados fundamentales del protocolo.
- No soporta **conexiones persistentes** ni **pipelining**, limitando la eficiencia en la transferencia de múltiples objetos.

HTTP/1.0 es un protocolo **no persistente**, es decir, requiere abrir una conexión TCP por cada recurso solicitado.



Este modelo es ineficiente para sitios con muchos objetos, ya que implica múltiples *round trips* y una sobrecarga importante sobre el sistema operativo y la red.

4.2 HTTP/1.1 — RFC 2616

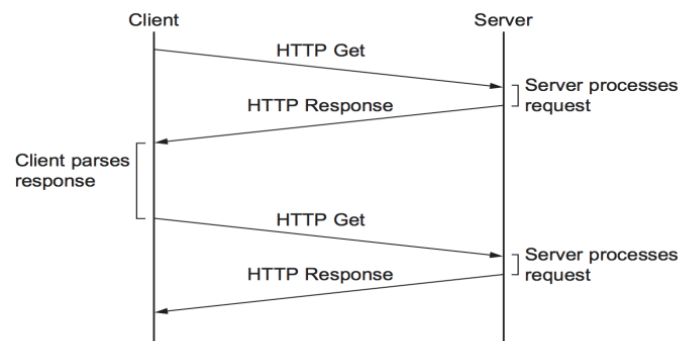
La versión **HTTP/1.1**, publicada en 1999 en el [RFC 2616](#), introduce mejoras significativas sobre HTTP/1.0 en términos de eficiencia, flexibilidad y control de caché.

4.2.1 Principales mejoras

- **Conexiones persistentes:** múltiples objetos pueden transmitirse sobre una misma conexión TCP.
- **Pipelining:** el cliente puede enviar varias solicitudes sin esperar las respuestas intermedias.
- **Encabezado Host obligatorio:** permite alojar varios sitios web en la misma dirección IP (virtual hosting).
- **Control avanzado de caché:** mediante Cache-Control, ETag, y If-Modified-Since.
- Soporte para nuevos métodos: PUT, DELETE, OPTIONS, TRACE.
- **Codificación de transferencia segmentada:** (Transfer-Encoding: chunked) permite enviar contenido dinámico.

4.2.2 Problemas de HTTP/1.1

Aunque HTTP/1.1 mejora la eficiencia mediante conexiones persistentes y pipelining, aún mantiene una limitación estructural: el servidor debe enviar las respuestas en el mismo orden en que recibió las solicitudes, lo que genera el problema conocido como **head-of-line blocking**.



4.3 HTTP Persistente y Pipelining

La persistencia busca reducir el número de conexiones TCP necesarias para cargar una página web. Sin persistencia, cada objeto requiere su propio ciclo de conexión TCP, cuyo costo aproximado es:

$$2 \times RTT \times \text{Objeto}$$

Esto implica una gran sobrecarga en el sistema operativo y congestiona la red.

HTTP Persistente

- Permite reutilizar una conexión TCP para múltiples solicitudes y respuestas.
- En HTTP/1.1, las conexiones son persistentes por defecto.
- Disminuye la latencia y mejora el rendimiento global.

Pipelining

- Sin pipelining, el cliente espera completar un pedido antes de enviar el siguiente.
- Con pipelining, el cliente envía múltiples solicitudes apenas detecta las referencias necesarias (comportamiento por defecto en HTTP/1.1).
- El servidor responde en el mismo orden, manteniendo el orden de las solicitudes.

4.4 Etiquetas HTML Relevantes

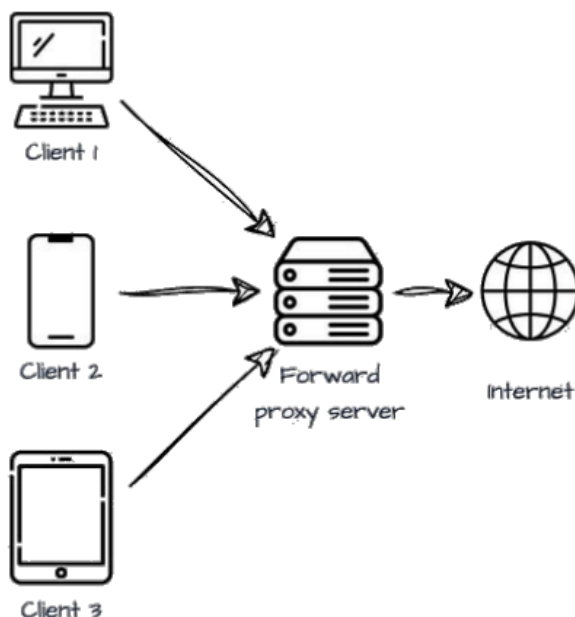
Ciertas etiquetas HTML generan automáticamente solicitudes HTTP para obtener recursos adicionales del servidor. Algunos ejemplos comunes incluyen:

- `<script>`: carga scripts externos (JavaScript).
- `<link rel="stylesheet">`: carga hojas de estilo CSS externas.
- `<form>`: al enviar el formulario, envía datos al servidor.
- `<a>`: realiza una nueva solicitud HTTP al navegar a otra URL.
- `<img>`, `<audio>`, `<video>`: cargan recursos multimedia a través del atributo `src`.

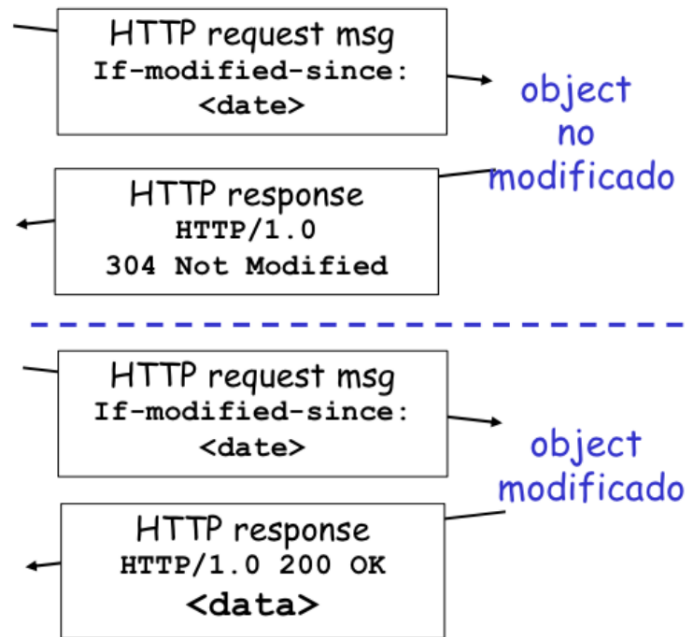
4.5 Técnicas de Cacheo en HTTP

El uso de caché es fundamental para reducir la carga del servidor y mejorar la velocidad de acceso del cliente.

Funcionamiento general



- El navegador puede configurarse para acceder a la web a través de un **proxy cache**.
- El navegador envía todas las solicitudes HTTP al cache intermedio.
- Si el objeto solicitado está en caché y es válido, el cache lo devuelve directamente al cliente.
- Si no está disponible, el cache solicita el recurso al servidor original, lo almacena y luego lo envía al cliente.



HTTP/1.1 proporciona encabezados específicos para controlar este comportamiento:

- Cache-Control: define políticas de almacenamiento (no-cache, max-age, etc.).
- ETag: identificador único del recurso.
- If-Modified-Since: permite al servidor responder con un código 304 Not Modified si el recurso no cambió.

5 Ejercicio 2

Suponga la siguiente página escrita en HTML que reside en el servidor `www.fcen.uba.ar`:

```

1 <html>
2   <head>
3     <title>Facultad de Ciencias Exactas y Naturales</title>
4     <link rel="stylesheet" type="text/css" href="style.css" />
5   </head>
6   <body>
7     <div>
8       
9       <a href="avsearch.php">  </a>
10    </div>
11    <div>
12      <form name="searchform" action="search">
13        <label>Buscar</label>
14        <input name="SearchableText" type="text" title="Buscar en el Sitio" />
15        <input type="image" src="search_icon.gif" />
16      </form>
17    </div>
18  </body>
19 </html>
  
```

¿Cuánto tiempo en términos de RTTs transcurrirá como mínimo, hasta transferir la totalidad de la información en HTTP/1.0?

Como es 1.0 entonces va a haber una conexión nueva para cada objeto que se deba traer del servidor y tomamos cada una como un RTT. Entonces tenemos :

- $2 \times RTT$ por el envío del documento HTML propiamente dicho.
- $2 \times RTT$ para traer la hoja de estilos CSS.
- $2 \times RTT$ para traer la imagen `searchline.png`.
- $2 \times RTT$ para traer la imagen `home.png`.

- $2 \times RTT$ para traer el ícono `searchicon.gif`.

`avsearch.php` no se trae inicialmente, se procesa su pedido cuando el cliente clickeee sobre el link.

¿Y en HTTP/1.1?

Comparativa de tiempos con y sin *pipelining*

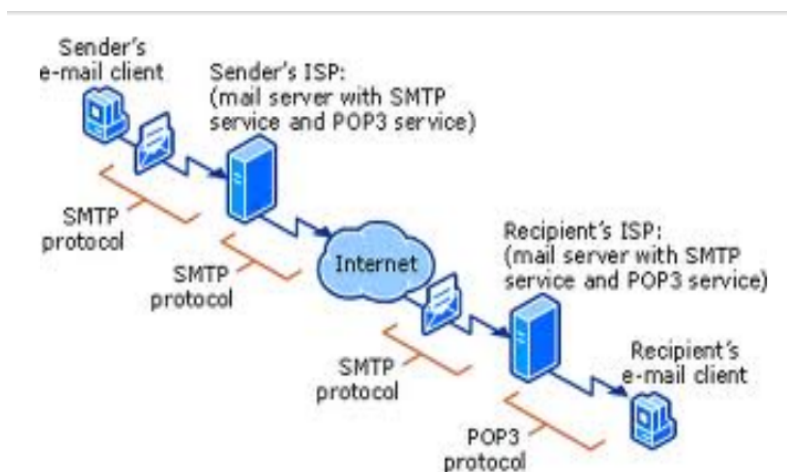
- En el caso de utilizar *pipelining*:
 - $1 \times RTT$ para inicializar la conexión.
 - $1 \times RTT$ para traer el documento HTML.
 - $1 \times RTT$ para traer todas las imágenes y hojas de estilo CSS.

Esto sucede así porque, al no cerrarse la conexión para cada pedido, se puede asumir que al usar *pipelining* la transferencia de los archivos incluidos en el HTML ocurre dentro de un único ciclo de *RTT*.

- En el caso de no utilizar *pipelining*:
 - $1 \times RTT$ para inicializar la conexión.
 - $1 \times RTT$ para traer el documento HTML.
 - $1 \times RTT$ por cada imagen y hoja de estilo CSS.

6 Correo Electrónico (SMTP, POP3, IMAP, Webmail, MIME)

El sistema de correo electrónico en Internet está compuesto por varios elementos que interactúan para permitir el envío y la recepción de mensajes.



Componente	Rol	Descripción
Servidor saliente	Envío	Encargado de transmitir los correos desde el cliente hacia otros servidores mediante el protocolo SMTP (RFC 2821).
Servidor entrante	Recepción	Almacena los mensajes recibidos. Los clientes acceden mediante POP3 (RFC 1939) o IMAP (RFC 3501).
Relay	Intermedio	Servidor que reenvía los mensajes entre distintos dominios o redes.
Interfaz de usuario	Cliente (User Agent)	Programa que permite al usuario redactar, enviar, recibir y visualizar mensajes (por ejemplo, Thunderbird o Webmail).

6.1 Protocolos de correo

Protocolo	Función	Características
SMTP	Envío	Protocolo simple de transferencia de correo entre servidores. Utiliza comandos como HELO, MAIL FROM, RCPT TO, DATA y QUIT. Las respuestas del servidor son códigos numéricos (2xx éxito, 5xx error, etc.).
POP3	Recepción	Permite descargar los mensajes del servidor al cliente. Normalmente elimina los correos del servidor una vez descargados. Protocolo definido en RFC 1939.
IMAP	Recepción	Permite acceder y gestionar mensajes directamente en el servidor sin descargarlos. Soporta carpetas, sincronización y estados de lectura. Definido en RFC 3501.
Webmail	Interfaz	Acceso a correos mediante un navegador web. Usa HTTP/HTTPS para comunicarse con un servidor que internamente utiliza SMTP/IMAP/POP3.

6.2 Ejemplo de flujo SMTP

```
S: 220 Servidor ESMTP
C: HELO miequipo.midominio.com
S: 250 Hello, please to meet you
C: MAIL FROM: <yo@midominio.com>
S: 250 Ok
C: RCPT TO: <destinatario@sudominio.com>
S: 250 Ok
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Subject: Campo de asunto
C: From: yo@midominio.com
C: To: destinatario@sudominio.com
C:
C: Hola,
C: Esto es una prueba.
C: Hasta luego.
C:
C: .
S: 250 Ok: queued as 12345
C: quit
S: 221 Bye
```

6.3 Formato del mensaje

Los mensajes de correo se estructuran en dos partes principales:

- **Encabezado (Header):** Contiene metadatos del mensaje. Ejemplos: **From:**, **To:**, **Subject:**, **Date:**, **Cc:**, etc.
- **Cuerpo (Body):** Contiene el contenido principal del mensaje, originalmente en texto plano ASCII.

6.4 Extensiones MIME (Multipurpose Internet Mail Extensions)

El estándar MIME ([RFC 2045](#)) amplía las capacidades del correo electrónico más allá del texto ASCII plano. Permite incluir contenido multimedia y adjuntos mediante codificación en ASCII.

Campo MIME	Descripción
MIME-Version:	Indica la versión del estándar MIME utilizada (por ejemplo, 1.0).
Content-Type:	Especifica el tipo de contenido (text/plain , text/html , image/png , multipart/mixed , etc.).
Content-Transfer-Encoding:	Define cómo se codifica el contenido para mantener compatibilidad ASCII (por ejemplo, base64 o quoted-printable).
Boundary	Delimitador para separar múltiples partes dentro de un mensaje multipart .

6.5 Ejemplo de extensiones al formato de mensaje

```
MIME-Version: 1.0
Content-Type: multipart/mixed;
boundary="-----417CA6E2DE4ABCAFB5"
From: Alice Smith <Alice@cisco.com>
To: Bob@cs.Princeton.edu
Subject: promised material
Date: Mon, 07 Sep 1998 19:45:19 -0400

-----417CA6E2DE4ABCAFB5
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

Bob,

Here's the jpeg image and draft report I promised.

--Alice

-----417CA6E2DE4ABCAFB5
Content-Type: image/jpeg
Content-Transfer-Encoding: base64
... unreadable encoding of a jpeg figure
```

6.6 POP3 e IMAP — Recepción de mensajes

Una vez que el correo llega al servidor entrante, el usuario lo recupera mediante alguno de los siguientes protocolos:

6.6.1 POP3 — Post Office Protocol v3

- Protocolo simple basado en texto definido en el [RFC 1939](#).
- Descarga los mensajes del servidor al cliente local.
- Típicamente, los mensajes se eliminan del servidor tras la descarga (aunque puede configurarse lo contrario).
- Opera sobre el puerto TCP 110.

6.6.2 IMAP — Internet Message Access Protocol

- Definido en el [RFC 3501](#).
- Permite acceder y gestionar los mensajes directamente en el servidor, sin descargarlos completamente.
- Soporta múltiples carpetas, sincronización entre dispositivos y búsquedas remotas.
- Opera sobre el puerto TCP 143.

Comandos básicos de IMAP:

Comando (Cliente)	Descripción / Respuesta (Servidor)
LOGIN	Autentica al usuario.
SELECT	Selecciona un buzón o carpeta.
FETCH	Recupera mensajes o encabezados específicos.
DELETE	Elimina un mensaje.
OK	Indica que la operación fue exitosa.
NO	Indica un error o rechazo temporal del servidor.
BAD	Indica un error de sintaxis o un comando inválido.

6.6.3 Webmail

El **Webmail** integra las funcionalidades del correo electrónico tradicional dentro del entorno HTTP, permitiendo acceder a los mensajes mediante navegadores web. En este modelo:

- El cliente utiliza HTTP/HTTPS en lugar de POP3 o IMAP.

- El servidor web actúa como intermediario entre el navegador y los servidores de correo.
- Ejemplos: Gmail, Outlook Web Access, Yahoo! Mail.

7 Ejercicio 3

Desde una PC se envía un correo a tdc-doc@dc.uba.ar y luego se accede al recurso web <http://www.exactas.uba.ar/index.html>. La PC se conecta a la web a través de un servidor proxy con dirección IP 208.190.1.22.

Enumere los mensajes SMTP que son necesarios para el envío de un mail a pedro@exactas.uba.ar desde un host en el dominio untref.edu.ar hasta que llega al servidor de destino.

Recordemos rápidamente cómo funciona el sistema de envío de correo electrónico:

- El cliente envía el correo a su servidor **SMTP**.
- Es este último el que se comunica con el **SMTP destino**, enviándole el mensaje.
- El correo ya llegó a destino. El hecho de que el receptor “baje los mails” es un tema aparte, asociado a **IMAP** o **POP3**.
- Flujo general: Host → SMTP origen → SMTP destino.

Con esa idea, veamos el orden de mensajes que se intercambian durante la entrega:

1. Comunicación Cliente–Servidor SMTP (saliente)

1. El cliente envía **HELO** y recibe del servidor saliente **250 Hello**.
2. El cliente envía **MAIL FROM: <usuario@untref.edu.ar>** y recibe del servidor saliente **250 Ok**.
3. El cliente envía **RCPT TO: <pedro@exactas.uba.ar>** y recibe del servidor saliente **250 Ok**.
4. El cliente envía **DATA** y recibe del servidor saliente **354 End data with <CR><LF>.<CR><LF>**.
5. El cliente envía el mensaje y finaliza la transferencia con **<CR><LF>.<CR><LF>**, recibiendo **250 Ok**.
6. Finalmente, el cliente envía **QUIT** y el servidor responde **221 Bye**.

2. Comunicación entre Servidores SMTP (relay o entrega final)

1. El servidor saliente envía **HELO** y recibe del servidor entrante **250 Hello**.
2. El servidor saliente envía **MAIL FROM: <usuario@untref.edu.ar>** y recibe **250 Ok**.
3. El servidor saliente envía **RCPT TO: <pedro@exactas.uba.ar>** y recibe **250 Ok**.
4. El servidor saliente envía **DATA** y recibe **354 End data with <CR><LF>.<CR><LF>**.
5. El servidor saliente envía el cuerpo del mensaje y finaliza con **<CR><LF>.<CR><LF>**, recibiendo **250 Ok**.
6. Por último, el servidor saliente envía **QUIT** y el servidor entrante responde **221 Bye**.

Detalle los mensajes HTTP (Requests y Responses) desde que la PC envía el primer GET hasta que le llega el respectivo Response.

Primero definimos el mensaje que se genera para el *request* HTTP:

```
GET /index.html HTTP/1.1
Host: www.exactas.uba.ar
```

Este mensaje es enviado desde el navegador web en la PC hacia el servidor **proxy**. Si el proxy tiene en su caché el recurso solicitado, responde directamente al usuario con los datos almacenados. En caso de no tenerlo en caché, el servidor proxy establece una conexión TCP con el servidor web correspondiente al nombre **www.exactas.uba.ar**, y este último responde con:

```
HTTP/1.1 200 OK
data
```

Donde **data** representa el contenido del recurso solicitado. Luego, la respuesta llega al proxy, quien la reenvía al cliente (la PC del usuario).